

Chapter 17

Appendix

17.1 Introduction

This appendix offers a comprehensive overview of the foundational IT infrastructure required to support modern GenAI and LLM workloads. It outlines how traditional computing environments—originally designed for batch processing, transactional workloads, and classical machine-learning models—must evolve to accommodate high-throughput, GPU-intensive, multi-modal AI systems. The discussion covers compute clusters, storage layers, networking, and security foundations, followed by guidance on upgrading these components to meet the demanding requirements of GenAI, such as distributed training, scalable inference, vector-optimized storage, and hybrid cloud deployment patterns. These elements form the baseline upon which reliable enterprise-grade AI capabilities can be built.

Beyond infrastructure, the appendix surveys the software frameworks, tools, and libraries that underpin LLMOps and RAGOps. It explains how orchestration frameworks, feature and embedding stores, experiment-tracking platforms, and multi-environment DevOps/ML governance pipelines support the lifecycle of GenAI models. Special emphasis is placed on RAG evaluation metrics—including factual accuracy, grounding consistency, retrieval quality, and latency—along with their practical implications for production systems. Serving and monitoring tools are also examined, covering model hosting platforms, vector-database performance dashboards, observability stacks, and drift-detection modules that help teams maintain model robustness and align outputs with business and regulatory expectations.

Finally, the appendix introduces several complementary components that complete the broader ecosystem, including API gateway technologies for secure access control, agentic-AI toolkits for building multi-agent systems, and lightweight utilities that support prompt management, cost tracking, safety enforcement, and auditability. These additions illustrate how GenAI platforms are becoming increasingly modular and composable. The appendix also outlines some emerging trends, such as self-optimizing retrieval pipelines and agent-driven RAGOps workflows, that may shape next-generation enterprise architectures.

17.2 IT Infrastructure Upgrade

IT infrastructure and application architecture need enhancement to implement LLMOps. For example, the computational power required to train LLMs, especially considering their size and complexity, is substantial. There is a general inadequacy in the current computational support available for training these models. Meanwhile, the microservice (API)-based architecture is critical for LLM gateways ^a. Figure 3 in Chapter 2 shows a reference logic architecture with the major functional layers.

17.2.1 The Application Layer

The Application Layer in LLMOps architectures can be effectively segmented into two main sub-layers, each specializing in different aspects of the application's functionality: Frontend & Backend.

The frontend serves as the primary user interface where users submit their queries and interact with the application. This layer can vary widely depending on the specific use case; for real-world applications, this might involve a web service interface designed for user-friendly interactions. For development and testing environments, a more controlled "playground"

^a Andreessen Horwitz, <https://a16z.com/emerging-architectures-for-llm-applications/>, <https://github.com/a16z-infra/llm-app-stack>, Jun 2023.

setup allows for the thorough testing of various LLM configurations under simulated conditions.

The backend of the Application Layer is where the core processing happens. It handles API requests initiated by the frontend, performing the complex tasks of interpreting user inputs and generating appropriate responses. This backend functionality integrates closely with services provided by LLMs and the Data Layer. Key components of the backend include:

- **Input Handler:** Manages the initial processing of user inputs.
- **Response Formation:** Constructs responses based on the processed inputs and integration with LLM outputs.
- **Output Handler:** Delivers the generated responses back to the user interface or further downstream processes.

Optionally, the backend may also interact with application-specific databases located in the Data Layer. These interactions facilitate the creation of Operational Stores, which preserve a record of the conversational history and user feedback. Additionally, Configuration Stores might be maintained to refine query handling and enhance the response generation process based on historical interactions and feedback. This structured approach ensures that applications built on LLMOps frameworks can efficiently handle user interactions while dynamically improving response quality and relevance through continuous learning and adaptation.

17.2.2 *Orchestration Layer*

Orchestration in LLMOps involves managing complex workflows that interact with multiple LLMs to automate and optimize various tasks. Unlike traditional operations that might call an LLM once for a single output, orchestration in LLMOps is about creating dynamic and intelligent systems that can loop inputs and outputs through different models or the same model multiple times. This cyclic querying can refine the context or adjust responses based on evolving data inputs or strategic needs. For instance, an initial output from one LLM might feed into another LLM to enhance understanding or generate a more nuanced response. This is

particularly useful in applications requiring sophisticated decision-making processes, such as automated customer support systems, where queries might need escalating or rerouting based on their complexity or specificity. Moreover, orchestration enables the parallel processing of queries, leveraging the capabilities of various LLMs simultaneously. This approach not only accelerates the response time but also enhances the robustness of solutions, providing a richer, more integrated response system. In practice, orchestration might involve a coordinator system that directs queries to specific LLMs based on the task at hand, e.g., language translation, content generation, or complex problem-solving. This system is designed to handle and optimize workflows, ensuring that resources are used efficiently and that outputs are synchronized across the process chain. The underlying architecture of an orchestrated LLMOps environment involves several functionalities, including load balancers, workflow managers, monitoring and service meshes, which help in routing requests and managing data flow among the LLMs. Effective orchestration aims to minimize latency, manage load distribution, and maintain system resilience against failures. Advanced techniques such as caching responses for recurrent queries and employing fallback mechanisms for handling processing delays are also integral to a well-orchestrated LLMOps framework. Ultimately, the goal of orchestration in LLMOps is to automate as much of the decision-making and integration process as possible, making the overall execution more efficient and adaptable to the needs of various applications.

Note that one main purpose that we put the monitoring and reporting as one component inside this layer is for observability, e.g.,

Infrastructure Monitoring: The monitoring of infrastructure, including hardware utilization, log collection, and metrics tracking, is efficiently managed using one or few tools, e.g., AWS CloudWatch. Those tools shall offer comprehensive insights into the operational performance of LLM deployments, enabling a thorough understanding of system dynamics and resource usage.

Model-Specific Observability: For model-specific observability (performance, efficiency and ethics, e.g., fairness and explainability), it calls the corresponding function in the LLM API layer. For example, we might consider integrate the tools, e.g., SageMaker Clarify in AWS,

NVIDIA NeMo-Guardrails, Guardrails by Guardrails.ai, LLM-Guard by ProtectAI, LLAMA GUARD (Purple Llama). Those tools or APIs provide developers with the capability to automatically evaluate LLMs using recognized metrics such as GLUE, ROUGE, and BLEU. The tools aid in enhancing transparency and understanding of model behaviors, contributing to more informed model improvement and validation efforts.

17.2.3 *LLM Layer*

The LLM Layer is a crucial component of the LLMOps architecture, functioning as the central hub for processing and managing language model requests. This layer is specifically engineered to address the intricacies and demands associated with handling large language models, thereby underpinning the core functionalities of the LLMOps.

At the heart of the LLM Layer is the LLM API Gateway. This gateway serves as the principal conduit for all incoming and outgoing requests, centralizing the processing activities. It features scalable integrations with a variety of LLM vendors, enhancing the system's adaptability and ensuring smooth service transitions as needed. Such flexibility is essential for maintaining operational efficiency and quickly adapting to shifts in technology or vendor capabilities.

One of the pivotal roles of the LLM API Gateway is its capacity to monitor and manage the costs related to the usage of LLMs, such as token generation and subscription fees. This capability is instrumental in controlling the operational budget and optimizing the allocation of resources. Moreover, the gateway maintains comprehensive logs of all interactions. These logs are not merely records but are valuable data sources that can be mined to gain insights for system enhancement, troubleshooting, and analyzing usage patterns.

Within the LLM service layer (mainly for inference), direct access is provided to both LLM models and Embedding models. The LLM models are fundamental to the LLMOps ability to comprehend and generate language, forming the backbone of its linguistic capabilities. Simultaneously, the Embedding models play a significant role by interfacing with the Data Layer to vectorize text, thus enriching the semantic search functionalities within the LLMOps. This dual-model

approach ensures that the LLMOps remains at the forefront of technological efficacy and application versatility.

The LLM utility layer includes the model training and fine-tuning requirements, as well as the tooling for agents and format conversions, e.g., text-to-voice or voice-to-text. The model hub could be part of the global model inventory / registry, which contains all metadata information about the models, and the users can get the up-to-date information and obtain the approval of specific model access.

17.2.4 Data Layer

The Data Layer in LLMOps architectures is a fundamental component designed to bolster "enterprise-specific" intelligence through advanced data management and processing techniques. This layer utilizes a vector store to host unstructured data, e.g., documents are broken down into manageable chunks along with their embeddings and relevant metadata, and feature store to house structured data.

To navigate the inherent limitations of LLMs, such as token limits which restrict the amount of data that can be processed at once, the Data Layer employs a "chunking" technique. This method divides long documents into smaller, more manageable segments, allowing for stepwise searching across the entirety of a document without exceeding the LLM's token limits. The metadata associated with each chunk enriches search capabilities, linking results back to their source documents and preserving the context necessary for accurate and relevant responses. The selection of an appropriate vector database and chunking strategy is crucial for optimizing data retrieval and processing. Strategies may vary between syntactic and semantic chunking or between variable and fixed segment sizes, each with distinct implications for how data is handled and utilized within the system.

To manage and process the vast amounts of data effectively, the raw data may be stored and manipulated in a Data Lake or Lakehouse ^b alongside

^b Potentially commercial frameworks, e.g., Databricks Delta table, Azure Data Lake Storage, AWS S3, Snowflake and Starburst Data Lakehouse or open-source frameworks like Hadoop, Iceberg and Apache Hudi.

data processing pipelines [°]. These pipelines are responsible for parsing, chunking, enriching metadata, and vectorizing data, subsequently populating the vector databases. Ensuring high data quality and robust governance is essential for maintaining well-structured and indexed data storage, facilitating efficient operations.

Moreover, the Data Layer can integrate various APIs, such as access control with Active Directory, to further enhance the overall functionalities, e.g., security. Data access control mechanisms ensure that information security and compliance are upheld, preventing unauthorized access to sensitive data and aligning with best practices for data privacy. These APIs can also add layers of context and pertinent information, significantly enriching the intelligence of responses.

In general, the Data Layer in LLMOps not only supports the functional requirements of handling large volumes of complex data but also enhances the intelligence and responsiveness of the Knowledge Assistant, making it a critical component in the broader architecture of LLMOps.

17.2.5 Infrastructure Layer

The Infrastructure Layer in LLMOps serves as the foundational backbone that supports the deployment, operation, and scaling of LLMs. This layer is typically engineered to handle the high computational demands of LLMs, facilitating efficient data processing, storage, and network management. Central to this layer are robust servers and data storage systems that ensure the rapid retrieval and processing of vast datasets necessary for training and running LLMs. Moreover, the infrastructure must be capable of dynamic scaling to accommodate varying workloads, which is critical in environments where the usage patterns can fluctuate significantly.

Cloud-based architectures are often utilized within the Infrastructure Layer due to their scalability and flexibility. These environments provide essential services such as managed hardware, virtualization, and high availability, which are pivotal for maintaining the continuous operation of

[°] Potential tools: kedro, Dagster, Luigi, Apache Airflow, Azure Data Factory, Google Cloud Dataflow, AWS Glue, etc

LLM applications. On-prem or hybrid structure is also employed in the financial industry. Additionally, advanced networking infrastructure ensures low-latency communications between distributed components of the LLMOps, crucial for real-time applications like interactive chatbots or live data analysis systems. The infrastructure also incorporates security measures to protect sensitive data and model integrity, which is paramount in preventing unauthorized access and ensuring compliance with data protection regulations.

The Infrastructure Layer must be meticulously designed and managed to ensure it can support the sophisticated requirements of LLM deployments, providing a stable, secure, and scalable platform from which the other layers of LLMOps can operate effectively.

17.3 LLMOps/RAGOps Implementation: An Architectural View

This section discusses two types of implementations: cloud-dependent and cloud-agnostic. In cloud-dependent architectures, platforms like AWS, Azure, and Google Cloud provide tightly integrated services for model deployment, monitoring, data management, and scalability. These architectures leverage native tools such as AWS SageMaker, Azure Machine Learning, and Google Vertex AI, offering seamless automation, CI/CD pipelines, and optimized infrastructure for running LLM or RAG applications. On the other hand, cloud-agnostic architectures focus on flexibility, allowing organizations to build AI systems that can be deployed across multiple cloud environments or on-premises. This setup uses interoperable components like observability tools, orchestration layers, vector databases, and model repositories to ensure portability, scalability, and operational consistency, independent of the underlying cloud provider.

17.3.1 *Cloud-dependent*

17.3.1.1 *AWS LLMOps Architecture*

This AWS architecture leverages a suite of Amazon Web Services to deploy and manage large language models effectively. It begins with Amazon API Gateway, which provides users with access to text and image generation models hosted on Amazon Bedrock, ensuring secure and scalable model availability across different environments. Automation is achieved through integration with AWS CodeCommit and AWS CodePipeline, facilitating continuous integration, delivery, and testing (CI/CD/CT) processes. This setup allows for dynamic updates and efficient management of code changes. For model training and workflow automation, Amazon SageMaker Pipelines are coupled with AWS Step Functions to streamline the fine-tuning of models, enhancing their performance and adaptability to specific tasks. The architecture also incorporates an Amazon OpenSearch Service cluster along with Amazon SageMaker Processing Jobs to automate the indexing of vector databases for RAG. This integration optimizes search functionality and improves the relevance of generated content within applications.

Furthermore, the AWS architecture supports the development of dynamic applications, such as non-player character dialogue systems in gaming, by providing deployment guidance and resources through repositories like GitHub. This comprehensive approach enables developers to build sophisticated AI applications with robust support for deployment and maintenance. The architecture ensures scalability, security, and efficiency, aligning with best practices in LLMOps.

17.3.1.2 *Azure LLMOps Architecture*

The LLMOps architecture in Azure is designed to streamline the operationalization of LLMs through a structured framework that emphasizes automation, monitoring, and collaboration. At its core is Azure Machine Learning, which supports the entire LLM lifecycle, from data preparation to model deployment and monitoring. Key components include Prompt Flow, which enables developers to create and manage workflows for LLM applications, and Azure Data Lake Storage, which facilitates efficient data management. Models are deployed and served via Azure Kubernetes Service (AKS) or Azure Container Instances, providing scalable and flexible environments. Integration with Azure Cognitive

Services enhances features such as natural language understanding and translation. Monitoring and MLOps practices, implemented using Azure Monitor and Azure DevOps, ensure continuous integration and delivery for iterative model improvements.

The architecture supports continuous integration and deployment (CI/CD) pipelines, automating the process of deploying LLM models and workflows as production endpoints. This ensures that models and prompt flows are easily accessible and regularly updated, contributing to overall system efficiency. Security and compliance are key pillars of the Azure LLMOps framework. Azure Key Vault secures sensitive data, while role-based access control (RBAC) manages resource access. These features ensure that the deployment of LLMs aligns with organizational policies and industry regulations, making it well-suited for enterprises in regulated sectors. Additionally, the architecture emphasizes continuous monitoring and evaluation to maintain model performance. Metrics like groundedness, relevance, and latency are tracked to ensure that deployed models meet business needs. Advanced techniques such as RAG can also be implemented to enhance LLM capabilities by dynamically integrating external knowledge sources. Throughout the framework, security and regulatory compliance remain paramount to protect sensitive data and ensure adherence to industry standards.

17.3.1.3 *Google Cloud Platform (GCP) LLMOps Architecture*

GCP's architecture is purpose-built to streamline the management and operationalization of LLMs through a flexible and modular approach. Vertex AI is a central component, managing all stages of the LLM lifecycle from data ingestion and preprocessing to model training and hyperparameter tuning. It integrates with Cloud Storage and BigQuery for large-scale data management, ensuring models are trained on vast datasets efficiently. Additionally, Vertex AI offers custom training jobs, enabling performance optimization through tailored model training and deployment pipelines. To serve models at scale, Vertex AI Endpoints provide low-latency inference, ensuring real-time responses. TensorFlow Extended (TFX) supports the entire machine learning workflow, offering features like data validation, transformation, and model evaluation. This makes it

easier for organizations to automate and manage their LLMs from start to finish, ensuring that models are robust and accurate in various applications.

For application deployment, Google Cloud Run plays a vital role by enabling the deployment of microservices to manage different parts of the LLM workflow. This serverless infrastructure ensures applications scale automatically with demand, significantly reducing operational costs during idle periods. Services such as Pub/Sub for message communication, Cloud Storage for data management, and Cloud SQL with pgvector for embedding storage enable smooth data ingestion, processing, and retrieval. The architecture is designed to integrate easily with users' data sources, such as URLs or Google Drive folders, enhancing flexibility. CI/CD processes are supported by Cloud Build and Cloud Functions, ensuring seamless updates to models. Tools like Explainable AI^d enhance monitoring and model transparency by providing insights into model predictions, aiding in debugging and maintaining compliance with industry standards. Security is reinforced by Identity and Access Management (IAM) and Cloud Audit Logs, ensuring that access to resources is properly controlled while complying with organizational policies. The architecture's strong focus on continuous monitoring, evaluation, and security ensures that organizations can maintain optimal performance, flexibility, and security as they scale their LLM applications on Google Cloud.

17.3.2 Cloud-Agnostic LLMOps Architecture

The cloud-agnostic architecture is designed to be platform-independent, allowing deployment across various cloud providers or on-premises environments. It incorporates components such as observability tools, LLM orchestration layers, APIs, semantic caching mechanisms, model repositories, feature stores, vector databases, and data platforms. These components are interconnected to facilitate efficient model inference,

^d It supports any TensorFlow model that can provide an embedding (latent representation) for inputs. Tree-based models, such as decision trees, and models from other frameworks, such as PyTorch or XGBoost, are not supported yet.

alignment, and training processes. Observability ensures continuous monitoring of the system's performance and health, enabling proactive maintenance and optimization. The LLM orchestration layer manages the flow of data and tasks between different components, ensuring smooth operations. Semantic caching improves response times by storing and reusing frequently accessed data or computations.

The architecture includes a model repository for versioning and managing different iterations of models, a feature store to manage and serve features consistently across training and inference, a vector database for retrieval operations essential for tasks like similarity search in RAG applications. This flexible architecture allows organizations to tailor components to their specific needs and integrate with various tools and services.

17.3.2.1 *Databricks LLMOps Framework*

This architecture is designed to efficiently manage LLMs by focusing on scalability, modularity, and integration with existing machine learning workflows. Key components include Delta Lake for data pipelines, MLflow for managing the model lifecycle, and Databricks Workflows to automate processes from development to production deployment. This framework ensures that organizations can seamlessly train, fine-tune, and deploy LLMs while maintaining scalability and operational efficiency across various production environments. This architecture has a “native” support for RAG, where vector databases enable fast similarity searches to enhance LLMs with contextually relevant data. Tools like Hugging Face Transformers and LangChain integrated with Databricks Runtime for ML allow users to fine-tune pre-trained models for specific applications, optimizing performance and reducing the need to develop models from scratch. The architecture also incorporates Mosaic AI Model Serving for easy model deployment and governance, facilitating smooth integration with external APIs or self-hosted solutions while ensuring robust model performance.

Additionally, Databricks emphasizes robust CI/CD practices using Git and MLflow for version control and experiment tracking. The platform's Lakehouse architecture, with its integrated storage and compute resources, enhances data governance and security, particularly in regulated

industries. Continuous monitoring through feedback loops ensures models remain relevant and improve over time, making Databricks' LLMOps a comprehensive solution for deploying and managing LLMs at scale.

17.3.2.2 *Ray Framework for LLMOps*

Ray's LLMOps architecture is designed to operationalize LLMs using a distributed and scalable framework. At its core is Ray Serve, which provides a robust infrastructure for model training, inference, and observability. Ray's architecture includes four key components: data engineering, model training, real-time inference, and observability. Ray Serve allows organizations to deploy models as microservices, supporting both single-node and multi-node deployments. This flexibility enables efficient management of inference workloads by distributing tasks across multiple nodes, ensuring scalability and high availability. Additionally, Ray's cloud-agnostic design allows integration into various environments, making it suitable for hybrid and on-premises setups.

The framework can be also organized into five primary layers as described earlier. The Data layer handles ingestion and preprocessing, while the LLM layer focuses on distributed model training and inference. The Orchestration layer manages task coordination, scheduling, and resource allocation, ensuring seamless operation across components. The Application layer supports end-user functionalities built on the underlying infrastructure, making Ray well-suited for scalable AI applications. Ray also supports some additional components for hyperparameter tuning (Ray tune), experiment management (e.g., integration with CometML or Weights & Biases), and explainability (e.g., integration with Seldon's Alibi). In addition, Ray provides a strong foundation and ecosystem for scaling LLMs and managing workloads efficiently [°]. Its powerful distributed computing capabilities, combined with flexible deployment options and robust observability tools, make it an effective solution for developing, deploying, and managing LLMs at scale. Organizations looking for a flexible, cloud-agnostic LLMOps framework can benefit

[°] See more libraries that have integrations with Ray at <https://docs.ray.io/en/latest/ray-overview/ray-libraries.html>

from Ray's ability to adapt to different environments and computational needs.

17.3.2.3 Open-source RAG Application Frameworks

Besides the general LLM application frameworks like LangChain, Llamaindex Haystack, and Hugging Face Transformers, there are some so-called “all-in-one” frameworks, such as

RAGFlow^f: It is an open-source framework built to streamline the development and deployment of RAG systems. It focuses on modularity and easy integration with a variety of data sources, including documents, images, and structured datasets. With RAGFlow, users can efficiently implement RAG pipelines, leveraging LLMs to provide grounded, context-aware responses by integrating external data sources. RAGFlow supports both cloud and on-premises deployments, making it flexible for various use cases. For these without UI requirements, FlashRAG^g is also a good start.

GraphRAG By Microsoft^h: GraphRAG organizes information into interconnected graphs to enhance retrieval and context understanding in RAG systems. This graph-based approach helps capture the relationships between different pieces of information, leading to more accurate and relevant LLM responses. It is particularly useful for applications that require in-depth, structured retrieval, such as legal document analysis, scientific research, and knowledge management. A more efficient implementation is LightRAGⁱ.

AnythingLLM^j: It is a flexible framework that allows developers to create customized RAG systems by integrating various retrieval and generation components. This framework supports integration with multiple APIs and data sources, making it ideal for a wide range of applications, from small-scale projects to large enterprise-level systems.

^f <https://github.com/infiniflow/ragflow>

^g <https://github.com/RUC-NLPIR/FlashRAG>

^h <https://github.com/microsoft/graphrag>

ⁱ <https://github.com/HKUDS/LightRAG>

^j <https://anythingllm.com/>; <https://github.com/Mintplex-Labs/anything-llm>

Its modular design enables developers to build dynamic pipelines tailored to specific use cases.

[Cognita by TrueFoundry](https://www.truefoundry.com/cognita)^k: It is an open-source framework from designed to simplify the development and deployment of RAG systems. It provides a modular, API-driven architecture that supports customization and scalability, allowing users to handle large datasets efficiently. Key features include incremental indexing, multiple query handling, autoscaling, and support for various vector databases like Qdrant and embedding models from OpenAI and Cohere. Cognita also offers a user-friendly UI, making it suitable for both technical and non-technical users to experiment and deploy RAG systems. In addition, it uses Langchain/Llamaindex under the hood, which means we can always integrate the latest modules/functions from these two popular tools.

[Weaviate Verba](https://github.com/weaviate/Verba)^l: An open-source RAG application designed to make RAG accessible to users without extensive technical expertise. It features a user-friendly interface for customizing RAG pipelines and interacting with generated outputs. It can connect either locally with Ollama and Huggingface or through LLM providers such as Anthropic, Cohere, and OpenAI.

[Semantic Kernel by Microsoft](https://github.com/microsoft/semantic-kernel)^m: It is an open-source project developed by Microsoft, designed to facilitate the integration of LLMs with existing software applications. The core idea behind Semantic Kernel is to provide a framework that allows developers to easily create, orchestrate, and customize workflows around LLMs by utilizing a variety of skills like prompt chaining, memory, and learning capabilities. The tool enables developers to embed LLM capabilities directly into their applications while maintaining flexibility and control. It is highly customizable and supports a modular approach, allowing users to combine different AI services and integrate seamlessly with other Microsoft tools like Azure AI and OpenAI. In essence, Semantic Kernel helps streamline LLM-based operations in enterprise or software development contexts.

^k <https://www.truefoundry.com/cognita>

^l <https://github.com/weaviate/Verba>

^m <https://github.com/microsoft/semantic-kernel>

AutoRAGⁿ: This tool automatically identifies the best RAG pipeline for your specific data and use case. With many RAG pipelines and modules available, manually testing them all is time-consuming and difficult. It simplifies this by automating the evaluation of different module combinations, helping you quickly find the optimal solution based on your own evaluation data. Try it now to streamline your search for the best RAG pipeline.

NeumAI by NeumTry^o: NeumAI is an open-source AI framework designed to offer a user-friendly platform for building and deploying ML/AI models. The focus of NeumAI is to simplify AI model development by providing intuitive tools that minimize the complexity of setting up training environments, managing data, and scaling machine learning operations. The framework is built to be accessible for both beginners and experienced developers, emphasizing easy-to-understand APIs and modular components. NeumAI supports deep learning models and offers features that help with model optimization, experiment tracking, and collaborative AI development. It is designed to be highly flexible and adaptable for various AI applications, from research to production environments.

Note that we only enumerate some of the LLMOps frameworks currently available in the market. In fact, there are many other tools and platforms designed for managing, deploying, and optimizing LLMs, e.g., ElasticSearch, Cohere, Domino Data Lab, etc. In my view, there is no one definitely better than another. From our experience, we initially used Cognita because its wrapper for Langchain and LlamaIndex makes it easy to integrate the latest modules and features from those two tools, especially when building a chatbot backend. We also extend the RAGFlow framework due to its drag-and-drop interface for designing agents, integration with GraphRAG /LightRAG, and its user-friendly documentation processing UI, making it accessible to non-technical business users. Users can opt to enable or disable GraphRAG, as constructing knowledge graphs can be resource-intensive. Additionally, AutoRAG helps in optimizing parameters within a simple pipeline. We

ⁿ <https://github.com/Marker-Inc-Korea/AutoRAG>

^o <https://github.com/NeumTry/NeumAI>

recommend experimenting with several frameworks to understand their strengths and limitations before choosing one for development. However, in this book, we mainly use Langchain and LlamaIndex to illustrate algorithms and techniques for simplicity, though these can be easily adapted to the other frameworks mentioned.

17.3.3 *Hybrid Infrastructure*

For highly regulated industry such as finance, hybrid structure using both on-prem and cloud is essential. Below is a simplified version to develop a LLMOps using such a hybrid infrastructure in a small scale.

Phase I: Architecture Design and Resource Preparation

In the first phase, you must design a hybrid infrastructure that strategically leverages both on-premises and cloud-based resources. Compact models such as 7- or 13-billion parameter foundational or quantized variants should be deployed locally to address sensitive or frequently recurring tasks, while more powerful models (30 B parameters and above) should run in the cloud or via commercial APIs from the likes of Qwen, DeepSeek, Gemma, OpenAI, or Anthropic. This hybrid solution achieves an optimal trade-off between cost and performance and can lower API expenses to a degree. You should also build a domain-specific knowledge base using vector-store solutions like Chroma, Elastic, pgvector or Pinecone, to RAG and boost semantic search accuracy to above 90%. In parallel, hardware must be provisioned with robust GPU clusters with at least 80 GB of memory per GPU and a minimum of 500 GB SSD storage to enable efficient model loading and distributed processing. Finally, implement a comprehensive security layer composed of an API gateway, traffic-rate limiting, sensitive-word filtering, and audit logging to ensure that any non-compliant content is intercepted with greater than 99.97% efficacy.

Phase II: Development and Fine-Tuning

Phase II focuses on creating a productive and secure environment for development and fine-tuning. Begin by segregating environments into development, staging, and production stages, injecting sensitive credentials via a secrets management solution such as AWS Secrets Manager and Azure Key Vault. Next, establish prompt engineering as an

industrialized process: design structured templates with fields such as `system_prompt`, `few_shot`, and constraints, using dynamic placeholders to inject user context and conversation history. For domain adaptation, employ parameter-efficient tuning methods like LoRA or QLoRA; these allow you to approximate the performance of full-parameter fine-tuning with just $< 1\%$ parameter increase, and in regulated domains such as legal or healthcare can deliver F1-score improvements exceeding up to 40% potentially.

Phase III: Deployment and Monitoring

In the third phase, deploy and observe your system with both performance and safety in mind. Choose inference engines that align with your deployment context: for example, use vLLM to support high-concurrency customer-service workloads on a single GPU or cluster via Ray; use Ollama for environments with limited hardware resources that still require support for many open-source models; and use TensorRT-LLM for applications demanding ultra-low latency, aiming for inference times under 50 ms. Concurrently, build an observation framework that continuously monitors three critical dimensions: factual correctness (implemented via real-time RAG validation against your knowledge base); cost efficiency (by tracking token usage combined with dynamic batch optimization); and safety (a multi-dimensional content scanning system that detects bias, toxicity, or hallucinations).

Phase IV: Continuous Iteration and Improvement

In the final phase, establish mechanisms for continuous improvement driven by real-world feedback. On the user side, instrument interaction touchpoints to collect metrics such as refusal rates and satisfaction scores. On the system side, automate evaluations of output quality using coherence and factuality scoring metrics. For controlled rollout of new model versions, adopt a canary deployment strategy that mitigates risk through gradual exposure. The typical implementation timeline spans from a 2-4 week proof-of-concept (PoC) phase and an 8-12 week minimum viable product (MVP) stage, through to a full enterprise rollout over 6-9 months. In sectors subject to strict regulation, add approximately 30% more time to allow for compliance assessments.

17.4 Reranking Algorithms

This section provides some detailed description on the reranking algorithms in Chapter 8.

- **Reciprocal Rank Fusion (RRF)**

Given a document d , the RRF score for document d is computed as:

$$RRF(d) = \sum_{\{i=1\}}^{\{n\}} \frac{1}{k + r_{\{d,i\}}}, \text{ where:}$$

- n is the number of ranked lists (retrieval sources),
- $r_{\{d,i\}}$ is the rank of document d in the i -th list (1-based indexing),
- k is a smoothing parameter (typically $k = 60$) to prevent dominance by top-ranked documents.

RRF is especially useful in multi-path retrieval pipelines, where different methods are used for initial candidate generation. After collecting candidate results from various retrievers, RRF can be used to:

- Efficiently select the top- N candidates for re-ranking,
- Reduce computational load by limiting the size of re-ranking input,
- De-duplicate and unify document sets from heterogeneous sources.

Since RRF is not dependent on the internal scoring mechanisms of retrieval models, it is simple, interpretable, and model-free, making it an attractive choice in production-grade hybrid search systems.

- **Maximal Marginal Relevance (MMR)**

Given a relevance-ranked list (e.g., from vector search), iterate the selection via the formulation below:

$$MMR(D_i) = \arg \max_{D_i \in R \setminus S} [\lambda \cdot \text{Sim}(D_i, Q) - (1 - \lambda) \cdot \max_{D_j \in S} \text{Sim}(D_i, D_j)],$$

where,

- Q denotes the user's query, and R represents the set of documents retrieved for Q .
- The set S includes those documents that have already been selected.

- The term $\text{Sim}(D_i, Q)$ quantifies how closely a document D_i relates to Q , while $\text{Sim}(D_i, D_j)$ captures the similarity between D_i and each previously selected document D_j .
- The parameter λ , ranging from 0 to 1, controls the trade-off between relevance and diversity: a larger λ places greater emphasis on relevance, while a smaller value favors diversity.

At each iteration, the algorithm selects the document D_i that maximizes this margin, thereby constructing a re-ranked list optimized for both relevance and diversity.

17.5 Evaluation Metrics

This section details the definitions and formulations for key evaluation metrics used in assessing RAG systems and LLM outputs in Chapter 10.

17.5.1 Statistical Metrics

- **ROUGE (Recall-Oriented Understudy for Gisting Evaluation)**

It is a suite of metrics commonly used in natural language processing to assess the quality of generated text, especially in summarization tasks. ROUGE-N measures recall of overlapping n-grams between the generated text (candidate) and reference text, focusing on how much of the reference's content is captured. ROUGE-L evaluates similarity by considering the length of the Longest Common Subsequence (LCS), thereby preserving sentence-level structure and capturing coherent, potentially non-consecutive matches.

$$ROUGE-N = \frac{\sum_{n\text{-gram} \in \text{Reference}} \text{Count}_{\text{match}}(n\text{-gram})}{\sum_{n\text{-gram} \in \text{Reference}} \text{Count}(n\text{-gram})}$$

$$\text{Precision}_{\text{LCS}} = \frac{\text{LCS}(X, Y)}{|Y|} \text{ and } \text{Recall}_{\text{LCS}} = \frac{\text{LCS}(X, Y)}{|X|}.$$

- **BLEU (Bilingual Evaluation Understudy)**

It is primarily used to evaluate machine-translated text by measuring the precision of n -gram overlaps. It calculates how much of the

generated candidate's n -grams appear in reference texts, and combines multiple n -gram precisions (usually up to four) using a weighted geometric mean. BLEU also includes a brevity penalty to penalize overly short outputs that might artificially inflate precision

$$BLEU = BP \cdot \exp \left(\sum_{n=1}^N w_n \log p_n \right), \text{ where,}$$

- p_n is the precision of n -grams, i.e., the proportion of n -grams in the candidate found in the reference(s).
- w_n are weights (often uniform, such as $1/N$).
- BP is the brevity penalty, which penalizes candidate texts shorter than the reference to avoid inflated precision scores.

- **METEOR Metric**

METEOR (Metric for Evaluation of Translation with Explicit Ordering) is an evaluation metric designed for translation tasks that takes into account synonyms, stemming, and word order. It provides a more nuanced assessment of the generated answer compared to the reference answer. By computing the harmonic mean of precision and recall (denoted as F_mean) and incorporating a penalty mechanism ($P_penalty$) to account for word order errors, METEOR is more effective at capturing semantic nuances than BLEU or ROUGE.

The METEOR formula is expressed as follows:

$$METEOR = F_{mean} \times (1 - P_{penalty})$$

Where:

- F_{mean} is the harmonic mean of precision and recall, used to comprehensively measure the degree of match between the generated output and the reference.
- $P_{penalty}$ is a penalty term applied to account for errors in word order and other discrepancies.

- **Precision, Recall, and F1 Score:**

- **Precision:**
Precision = True Positives / (True Positives + False Positives)

- **Recall:**
Recall=True Positives/(True Positives+False Negatives)
- **F1 Score:** $F1 = 2 \times \text{Precision} \times \text{Recall} / (\text{Precision} + \text{Recall})$

Metric	Normalized Discounted Cumulative Gain (NDCG)	Discounted Cumulative Gain (DCG)	Mean Reciprocal Rank (MRR)
Description	Measures ranking quality by comparing the DCG of a result list to the ideal DCG, allowing for normalized comparisons across queries.	Assesses the usefulness of a document based on its position in the result list, assigning higher scores to relevant documents appearing earlier.	Evaluates the average of the reciprocal ranks of the first relevant result across multiple queries.
Calculation Method	$NDCG = DCG / IDC$, where IDC is the maximum possible DCG for the ideal ordering of documents.	$DCG = rel_1 + \sum (rel_i / \log_2(i+1))$ for $i = 2$ to p , where rel_i is the graded relevance of the result at position i .	For each query, compute the reciprocal of the rank at which the first relevant document appears ($1/\text{rank}$). Then, average these reciprocal ranks over all queries.
Typical Use	Widely used in	Effective in	Suitable for

Cases	search engines and recommendation systems to evaluate ranking quality across diverse queries.	scenarios where multiple levels of relevance are assigned to documents, such as graded relevance assessments.	scenarios where only the position of the first relevant result is of interest, such as question-answering systems.
Strengths	Facilitates comparison across different queries; accounts for both the position and graded relevance of documents.	Considers the position and graded relevance of all retrieved documents; penalizes relevant documents appearing lower in the ranking.	Simple to compute and interpret; emphasizes the importance of retrieving the first relevant result promptly.
Limitations	Requires predefined relevance judgments; the computation of IDCG can be complex in large datasets.	Not normalized; DCG values are not directly comparable across different queries or datasets.	Does not account for multiple relevant documents; ignores the relevance of documents beyond the first correct one.

Table 1 comparison of some retrieval metrics

17.5.2 LLM Metrics

- **Coherence**

Coherence refers to the logical flow and consistency within a piece of generated text. It captures how well sentences and ideas connect to form a unified, sensible narrative, akin to the storyteller's thread that guides a keynote speaker, with each point building naturally on the previous one. In domains like finance, coherence ensures that a wealth manager's client profile narrative seamlessly bridges past investment behavior with future strategy, rather than presenting fragmented, disjointed insights.

Coherence = $\frac{1}{n-1} \sum_{i=1}^{n-1} \text{logical_connection}(s_i, s_{i+1})$, where,

s_i represents the i -th sentence, and n is the total number of sentences. The function $\text{logical_connection}(s_i, s_{i+1})$ quantifies how coherently sentence s_{i+1} follows sentence s_i .

- **Relevance (Response Relevancy & Answer Relevance)**

Relevance evaluates how accurately (and efficiently) the generated content addresses the given prompt, focusing specifically on its pertinence to the topic or question posed. Answer relevancy in RAG measures the precision and appropriateness with which the generated text directly addresses a specific query or question. Imagine an AI as a courtroom lawyer: every argument must directly address the judge's question. When asked for a loan agreement clause, the model shouldn't wander into unrelated fee structures. We quantify this by tracking how many key terms from the query appear meaningfully in responses, ensuring 90%+ focus on the task.

Response Relevance = $\text{relevant_content_words} / \text{total_content_words}$
or $\text{relevant_content_sentences} / \text{total_content_sentences}$ or embedding-based similarity

Answer Relevancy = $\text{number_of_relevant_answers} / \text{total_number_of_answers}$

- **Fluency**

Fluency assesses the grammatical correctness, naturalness, and readability of the generated text. For example, it is the difference between a polished TED Talk and a garbled voicemail. A banking compliance report riddled with awkward phrasing like "the client not

has submitted documents timely” erodes trust. Our systems flag such errors by comparing outputs to grammatically pristine financial documents.

$Fluency = \frac{1}{N} \sum_{i=1}^N f(s_i)$, where,

$f(s_i)$ can be a binary function $\in \{0,1\}$ or a probability function $\in [0,1]$. Both of them can be decided by an LLM-as-judge approach, as the model output, e.g., the former can be designed as binary acceptable or not, and the latter directly output the probability. The former can be also assessed via human ratings or acceptability classifiers (e.g., CoLA-style). Another error-rate-based formula aligns with GEC practice (ERRANT/M2/GLEU) can be expressed as

$Fluency_{err} = 1 - \frac{E}{T}$, where,

E is the total number of grammatical errors detected (e.g., via ERRANT) and T the total token count.

- **Adherence to Context**

Adherence measures how strictly the generated text follows the provided context, format, and instructions. Like a chef following a recipe, the AI must respect constraints. If tasked with summarizing a credit report in bullet points, it shouldn’t revert to paragraphs. We assess this by checking if responses match specified formats and tones, which is critical when generating formal SWIFT messages versus casual client emails.

Adherence to Context = $\frac{\text{context_adhering_sentences}}{\text{total_number_of_sentences}}$
 Adherence to Context = $\frac{\text{total_number_of_sentences}}{\text{context_adhering_sentences}}$

Here "context-adhering" means, e.g., correct format, separation style, tone, or other constraints. And whether adherence is judged by human raters, classifiers, or an LLM-as-judge.

- **Faithfulness**

Faithfulness evaluates the accuracy and reliability of the generated text, ensuring the content truthfully represents the source material or factual basis. It’s an important metric to evaluate the hallucination. For example, assume the model acts as a journalist paraphrasing source material without distorting facts. A single misstated interest rate in a mortgage summary

could trigger regulatory action. We cross-verify claims against original documents, penalizing any deviation from source data.

$$\text{Faithfulness} = \frac{\text{number_of_accurate_statements}}{\text{total_number_of_statements}} \quad \text{or} \quad \frac{\text{Total number of claims in the answer}}{\text{Number of claims in the generated answer supported by context}}$$

We may use entailment or Natural Language Inference (NLI) models or LLMs to test whether claims are supported (entailed), contradicted or neutral with respect to the context. Some variant strategies including F1-style scoring (precision vs. coverage), and partial-support weighting.

- **Self-Knowledge**

Self-knowledge evaluates the model’s ability to recognize and articulate its limitations, especially when uncertain about a response ^P. This metric captures the core of *intellectual humility* or *metacognitive awareness*. The mark of true intelligence is knowing what you don’t know. When queried about niche regulations like Botswana’s new crypto laws, a responsible system responds “*This exceeds my current knowledge: let me connect you with our international compliance team*” rather than guessing. We track how often models appropriately acknowledge uncertainty.

$$\text{Self-Knowledge} = \frac{\text{number_of_uncertainty_indications}}{\text{total_number_of_uncertain_queries}}$$

Usually, we use prompt to ask the model to self-evaluate and calibrate the number, e.g., after the model drafts an answer, prompt it to produce a scalar confidence like “P(True)” for its own answer; show a band (e.g., High/Medium/Low) rather than a raw percent. We can also use *self-consistency* as an uncertainty signal, e.g., generate k answers; let $\hat{p}$ be the fraction voting for the modal answer. Report bands like: High (≥ 0.8), Medium (0.5–0.8), Low (< 0.5). In RAG, we shall verify claims against evidence and lower confidence when support is weak, e.g., 3/5 claims are strongly supported by the provided documents; the rest are uncertain.

- **Fairness and Harmlessness**

^P A recent scientific study highlights that “LLMs may [to some extent] possess an internal mechanism that is reflective of self-knowledge,” demonstrating calibrated uncertainty and the ability to distinguish answerable from unanswerable questions [5].

Ensuring fairness in large language models (LLMs) poses unique challenges compared to traditional AI/ML systems. LLMs are trained on vast and diverse datasets, which often include complex, subtle, and sometimes unintentional biases that can be deeply ingrained in the model's learned representations. Consequently, LLMs are at risk of perpetuating or even amplifying these biases, particularly against marginalized groups. Addressing fairness in LLMs requires a comprehensive approach that encompasses the following key elements: identifying the sources of bias, developing fairness metrics, and implementing mitigate strategies. We might detect toxicity, stereotypicality, and other types of bias using some tools, e.g., Truelens, giskard, and wefe.

These metrics form a diagnostic toolkit, not just scoring AI performance, but shaping systems that enhance rather than replace human financial expertise. When bankers can't distinguish between AI-generated reports and those crafted by their top analysts. In practice, these metrics may be combined to produce an overall quality score, and iterative improvements to the prompts can be quantitatively measured by correlating LLM-judge scores with human judgments. Table 2 provides a summary on the metric selection, where the metrics Noise Sensitivity and Hallucination shall be the lower the better. Self-knowledge metrics depends on users' preference to decide if it is lower the better.

Category	Metrics
Overall Evaluation	- BLEU Score: Assesses the overall quality of generated text through n-gram overlap. - ROUGE Score: Measures summarization quality. - Exact Match: Checks for a perfect match between generated and reference responses. - String Presence: Evaluates whether expected strings appear in the output. - Precision and Recall: Indicates the correctness and completeness of the response
Retrieval Evaluation	- Context Precision: Measures the proportion of retrieved context that is relevant. - Context/Claim Recall: Evaluates the completeness of the retrieved context.

	- Context Entities Recall: Assesses the retrieval accuracy for key entities in the context. - Non-LLM String Similarity: Compares retrieval outputs with expected strings outside of LLM evaluation.
Generation Evaluation	- Response Relevancy: Determines how well the generated answer addresses the query. - Faithfulness: Checks if the generated content remains true to the source or reference. - Factual Correctness: Evaluates the accuracy of the facts in the output. - Noise Sensitivity: Quantifies the impact of irrelevant or noisy data on retrieval performance. - Semantic Similarity: Measures the closeness in meaning between the generated output and the reference answer. - Self-Knowledge: Evaluates the model's ability to recognize and articulate its limitations. - Hallucination: Measures the model's incorrect information generated that's not in the context.

Table 2 Evaluation metric selection

17.6 Serving and Monitoring Tools

Name	Description	Key Features	License	Comments
Braintrust Data	An integrated AI product stack that streamlines the development, evaluation, and deployment of AI	- Integrated evaluations and prompt playgrounds - Continuous integration	Proprietary	Focused on full-stack AI product development.

	applications, including featuring evaluations, prompt playgrounds, CI, dataset management, and API gateway.	for AI workflows - Unified API for multiple AI models		
<u>Arize AI / phoenix</u>	An observability platform focusing on real-time performance monitoring, error analysis, and debugging, providing actionable insights into model behavior for both LLMs and supervised ML.	- Model performance visualization - Drift & data quality issue detection - Integration with Weights & Biases	Elastic License 2.0 (source available with restrictions)	Best suited for monitoring and troubleshooting AI models.
<u>Weights & Biases</u>	An MLOps platform to streamline ML workflows with powerful tools for experiment tracking, model versioning, and collaborative development.	- Experiment tracking - Model versioning - Collaboration tools	Proprietary	Widely used in the ML community for comprehensive experiment management.

<u>MLflow</u>	A platform to streamline ML development.	- Experiment tracking - Model registry - Deployment tools	Apache 2.0 (Open Source)	Very popular and broadly adopted across many industries.
<u>PromptLayer</u>	A platform for tracking, managing, and sharing LLM prompt engineering.	- Prompt versioning - Performance analytics - Collaboration features	Proprietary	Useful for teams focusing on prompt management and iterative improvement.
<u>Helicone</u>	An open-source observability platform for LLMs. SaaS	- Cost-effective monitoring - User-friendly interface - Easy integration	Proprietary / Apache-2.0	A strong choice if you want to self-host LLM monitoring.
<u>Quotient AI</u>	A platform for evaluating AI products on real-world use-cases during research, development, and production.	- Real-world scenario evaluations - Continuous monitoring - Performance benchmarking	Proprietary	Designed for practical evaluation in real-world settings.

		ng		
<u>Portkey AI</u>	A platform (AI gateway) to develop, launch, maintain, and iterate generative AI apps and features. SaaS	- Rapid development tools - Maintenance & iteration support - Focus on generative AI	Proprietary / MIT	Emphasizes the full lifecycle of generative AI applications.
<u>Freeplay AI</u>	A platform to prototype, test, and optimize LLM features for customers. SaaS	- Prototyping tools - Testing frameworks - Optimization features	Proprietary	Geared toward rapid prototyping and iterative testing.
<u>Gentrace</u>	An API and SDK for evaluating and observing generative data.	- Comprehensive evaluation methods - Production data monitoring - SDK support	Proprietary / SDKs are MIT-licensed	API-first approach for production monitoring and evaluations.
<u>Patronus AI</u>	An automated evaluation and benchmarking platform for LLMs. SaaS	- Automated benchmarking - Real-	Proprietary	Focused on automating evaluations in real-world

		world scenario testing - Scoring tools		settings.
<u>Autoblocks AI</u>	A collaborative cloud-based workspace for rapid iteration on GenAI products.	- Collaborative workspace - Rapid iteration support - Provider-agnostic environment	Proprietary	Designed for teamwork on fast-changing GenAI projects.
<u>Context AI</u>	Tools for pre-launch LLM evaluations and post-launch analytics. SaaS	- Pre-launch evaluations - Post-launch analytics - User conversation analysis	Proprietary	Supports evaluation both before and after deployment.
<u>E2b dev</u>	Services to deploy, test, and monitor AI agents, featuring a secure, long-running cloud sandbox.	- Secure sandbox environment - Deployment & testing services - Internet access for LLMs	MIT-licensed	Geared toward deploying and safely testing AI agents.

<u>AgentOps</u>	A toolkit for evaluating and developing AI agents. SaaS	- Agent development tools - Monitoring capabilities - Replay analytics	Proprietary / MIT-licensed open-source app/SDK	Useful for teams focused on agent-based systems.
<u>ZenoML</u>	An AI evaluation platform enabling data visualization, model performance analysis, and interactive report creation.	- Data visualization - Performance analysis - Interactive reports	Proprietary / MIT	Report-focused solution for performance insights.
<u>Baserun</u>	Tools for model configuration, a prompt playground, monitoring, and prototype workflow.	- Model configuration - Prompt playground - End-to-end testing	Proprietary	Offers full visibility into LLM workflows and testing.
<u>WhyLabs</u>	An AI observability platform including LLM monitoring, guardrails, and security for ML and GenAI	- Observability tools - Guardrails implementation - Security features	Proprietary / LangKit <u>Apache-2.0 license</u>	Focused on safety, compliance, and operational monitoring.
<u>Log10</u>	An AI-powered	- Automatic	Proprietary	Emphasizes

	LLMOps platform that automatically optimizes prompts and models.	prompt optimization - Built-in logging & debugging - Fine-tuning capabilities	ry	optimization and continuous improvement.
<u>PromptFlow</u>	An open-source evaluation framework for LLMs with support for model/prompt/RAG eval, dataset generation, and self-hosting.	- Open-source framework - Multiple evaluation methods - Self-hosting capabilities	MIT	Flexible and developer-friendly for custom evaluations.
<u>Parca AI</u>	A platform and SDK for AI engineers providing tools for LLM evaluation, observability, and an enhanced prompt playground.	- SDK for AI engineers - Version-controlled prompt playground - Observability tools	Proprietary	Tailored for technical teams and engineering workflows.
<u>Galileo</u>	A platform for evaluation, fine-tuning, and real-time observability with high-	- Evaluation & fine-tuning tools - Real-time observability	Proprietary	Focused on improving output quality by reducing hallucinations

	accuracy hallucination guardrails.	- Hallucination guardrails		ns.
--	------------------------------------	----------------------------	--	-----

Table 3 RAG/LLM tools

17.7 API Gateway Tools

Solution	Key Features	License	Platform / Language	Notes
Kong Gateway	High performance; extensive plugin ecosystem; rate limiting; load balancing; authentication	Apache License 2.0 (OSS)	Nginx/ Lua/C (cloud - native)	Widely adopted with deployments in containerized and cloud environments.
Tyk API Gateway	Full-featured gateway with rate limiting, authentication, quotas, and an integrated developer portal	AGPL (Open Source)	Go	Known for its strong open-source roots and extensive customization.
Kraken D	High throughput; aggregation and transformation of multiple microservices; caching; rate limiting	Apache License 2.0	Go	Designed specifically for microservices architectures with an emphasis on performance and efficiency.

Apache APISIX	Dynamic configuration; load balancing; rate limiting; multi-protocol support; extensible with plugins	Apache License 2.0	Lua/Nginx (Open Resty)	A cloud-native gateway that supports advanced features and scalability.
Portkey -AI Gateway	Unified API for many LLMs; automatic retries; integrated guardrails; load balancing	MIT License	TypeScript/Node.js	Designed specifically for routing and managing LLM API calls, making it easy to switch between commercial and self-hosted models without code changes.
Helicone Gateway Fallbacks	Unified endpoint for multiple LLM providers; automatic fallback switching; error-based triggers; observability features	(Open-source; details in OSS docs)	Multi-language (via REST APIs)	Part of the Helicone OSS LLM observability suite; enables seamless switching among supported providers.
Neutrinoo API Gateway	Optimized inference engines in model selection and query route; Lightweight and extensible; supports rate	Commercial	Multiple SDK	An emerging, simple solution focused on providing fallback capabilities across different endpoints.

	limiting, request transformation, and retries			
OpenRouter	Unified interface for commercial and self-hosted LLMs to identify the most cost-effective option for each model from a range of providers; transformation and retry logic; rate limiting	Commercial	Multiple SDK	Frequently mentioned in conjunction with LLM integration tools; helps standardize access to mixed API endpoints, making it easier to swap or combine providers without code changes.

Table 4 API gateway solutions

17.8 Agentic AI Tools

Many agent frameworks and tools are available in the markets from the general-purpose agents, e.g., openai agents ⁹, langchain/langgraph and llamaindex, to specific agents, e.g., CrewAI, Phidata, Letta, Parlant and AutoGen.

Langchain Agent

Built on a flexible architecture, Langchain agents combine LLM-based planning with external tool integration, enabling them to perform actions like web searches, API calls, or database queries. The agent workflow typically follows a reason-action-feedback loop: the LLM generates a reasoning step (e.g., "To answer this, I need to search for X"), selects a tool, processes the output, and refines its approach. This modular design supports both short-term memory (in-context learning) and long-term

⁹ <https://developers.openai.com/tracks/building-agents/>

memory (vector stores), allowing agents to retain context and improve over time.

A key strength of LangChain Agents lies in their extensible tool integration, which connects LLMs to real-world data and functionalities. Developers can equip agents with prebuilt tools (e.g., calculators, search APIs) or custom tools for domain-specific tasks. The framework also supports multi-agent systems, where specialized agents (e.g., document analyzers, research assistants) collaborate under a meta-agent's coordination. This enables sophisticated workflows like Agentic RAG, where agents retrieve, process, and reason over external knowledge. By blending autonomous planning with human oversight, LangChain Agents balance adaptability with control, making them ideal for applications requiring structured problem-solving from data analysis to conversational AI.

LlamaIndex Agent

LlamaIndex (formerly GPT Index) focuses on bridging large language models with structured data sources, emphasizing a data-first approach to retrieval and understanding. Its architecture supports both in-context (short-term) memory and external storage solutions (long-term memory), enabling the model to maintain context while querying or reasoning over large volumes of data. By structuring indices across various data repositories (such as documents, databases, or APIs) LlamaIndex can rapidly fetch and interpret relevant information. This method of layered retrieval enhances the model's ability to provide nuanced, contextually accurate responses and fosters more engaging, human-like interactions.

Much like other agentic frameworks, LlamaIndex can be extended with specialized components or “agents” to tackle tasks such as document retrieval, Q&A, and summarization. These agents collaborate under a coordinating meta-agent, which orchestrates query planning, tool usage, and iterative reasoning. By modularizing the system in this way, organizations can seamlessly plug in additional data sources or NLP pipelines, unlocking a robust, dynamic workflow. This hierarchical approach strengthens the ability to manage complex tasks that require strategic planning and step-by-step decision-making, ultimately boosting the overall effectiveness of the system.

CrewAI

CrewAI is an open-source framework for orchestrating collaborative teams of AI agents that transcend traditional chatbot capabilities. By enabling agents to share knowledge, delegate tasks, and work collectively, CrewAI solves complex challenges through simulated team dynamics, mirroring roles and responsibilities found in human teams (e.g., developers, analysts). The platform provides organizations with a scalable structure to design multi-agent workflows, supporting sequential, hierarchical, or asynchronous collaboration models. Users define each agent's role, goals, and even backstories, fostering context-aware interactions that enhance productivity, decision-making, and innovation.

To optimize performance, CrewAI integrates advanced interaction protocols, including role-playing, task prioritization, and tool utilization. The framework embeds safety guardrails and memory systems, allowing agents to learn from past interactions and operate ethically. When paired with state-of-the-art language models (e.g., Groq's Llama3-70B), CrewAI supercharges content generation and task execution. By unifying these features, the platform helps organizations streamline operations, reduce redundancy, and achieve coordinated outcomes, i.e., transforming AI from isolated tools into synergistic teams.

Table 5 below is a brief table summarizing the key aspects of each framework:

Framework	License	Multi-Agent Support	Extensibility	Integration Capabilities	Primary Focus
LangChain	MIT License	Yes	High	Integrates with various LLMs and tools	Building LLM applications and agents
AutoGPT	MIT License	Implicit (Autonomous)	Via Plugins	Internet access, plugin system	Autonomous task execution

Langroid	MIT License	Yes	High	Message-based agent communication	Simplifying multi-agent LLM applications
AutoGen	Apache License 2.0	Yes	High	Supports various LLMs	Multi-agent LLM applications
OpenAgents	Apache License 2.0	Yes	Yes	AI tool integration	Hosting and sharing language agents
LlamaIndex	Apache License 2.0	Not Primary Focus	High	Data sources, databases	Data indexing and retrieval for LLMs
GPT Engineer	Apache License 2.0	No	Customizable	Supports various programming languages	Automated code generation
DemoGPT	MIT License	No	Limited	Generates Streamlit apps	Generating interactive applications
GPT	MIT	Not	Customiz	Internet	Autonom

Researcher	License	Specified	able	access for research	ous online research
AgentVerse	Apache License 2.0	Yes	High	Supports different LLMs	Multi-agent deployment
Agents	MIT License	Yes	High	Tool usage, web navigation, APIs	Building autonomous language agents
BMTools	MIT License	No	High	Tool integration with LLMs	Extending LLMs with tools
crewAI	MIT License	Yes	Yes	Automation capabilities	Agent framework for engineers
Phidata	MPL-2.0	Yes	Function Calling	API integration	Building AI assistants using function calls

Table 5 Comparison of agentic tools

Framework	Description	Key Features
LangChain	LangChain is a powerful framework for building applications and agents	- Modular Components: Offers prompt templates, chains, agents, and memory

	powered by large language models (LLMs). It provides a suite of tools and abstractions that simplify the development of complex LLM-based applications.	components. - Integration: Supports various LLM providers like OpenAI, Hugging Face, and more. - Memory Management: Tools for handling conversational context and state. - Extensive Documentation: Rich resources and examples for developers.
AutoGPT	AutoGPT is an experimental open-source application showcasing the capabilities of GPT-4. It enables developers to build autonomous AI agents that can perform tasks by chaining LLM "thoughts" together.	- Autonomous Task Execution: Operates without human intervention. - Internet Access: Gathers real-time data from the web. - Memory Management: Handles both short-term and long-term memory. - Plugin System: Extensible through community-developed plugins.
Langroid	Langroid streamlines LLM application development through multi-agent systems, where agents interact as primary components via message-passing collaboration.	- Multi-Agent Collaboration: Facilitates communication between multiple agents. - Message-Based Communication: Agents interact through defined messaging protocols. - Simplified Coordination: Eases the complexity of managing multiple agents.

AutoGen	AutoGen enables task-solving LLM applications by orchestrating conversations between multiple intelligent agents. It streamlines the creation of complex AI workflows involving agent collaboration.	- Multi-Agent Conversations: Agents communicate to achieve objectives. - Task Orchestration: Simplifies complex task management. - LLM Support: Compatible with various large language models.
OpenAgents	OpenAgents is an open platform for using and hosting language agents in the wild. It aims to facilitate the sharing, deployment, and collaboration of AI agents across different environments.	- Agent Hosting Platform: Allows users to deploy and share agents. - Community-Driven: Encourages collaborative development and sharing. - Tool Integration: Supports integration with various AI tools and services.
LlamaIndex	LlamaIndex is a data framework for connecting custom data sources to LLMs. It provides tools for indexing and querying data using LLMs.	- Data Ingestion and Indexing: Handles various data formats and sources. - Query Interface: Enables complex querying capabilities with LLMs. - Integration: Works with databases, APIs, and other data sources. - Scalability: Supports large datasets efficiently.
GPT	GPT Engineer is an open-source project that automates code	- Automated Code Generation: Produces code from natural language

Engineer	generation to complete development tasks. It leverages LLMs to generate and refine code based on user specifications.	prompts. - Customizable Workflows: Allows tailoring prompts and development processes. - Iterative Development: Supports refining code through multiple iterations.
DemoGPT	DemoGPT is an autonomous AI agent that creates interactive Streamlit applications. It generates app code based on user prompts, facilitating quick development of interactive demos.	- Code Generation: Automatically writes Streamlit app code. - Interactive Apps: Supports the creation of interactive user interfaces. - Ease of Use: Simplifies the process of app development with AI assistance.
GPT Researcher	GPT Researcher is an autonomous agent designed for comprehensive online research. It automates the process of gathering, summarizing, and analyzing information from the web.	- Autonomous Web Browsing: Navigates websites to collect data. - Information Summarization: Condenses large amounts of information. - Data Aggregation: Compiles data from multiple sources.
AgentVerse	AgentVerse streamlines the implementation of multi-agent LLM systems across diverse applications, offering comprehensive tools for behavior customization, coordination, and	- Multi-Agent Management: Handles multiple agents simultaneously. - Agent Coordination: Defines interactions between agents. - LLM Support: Compatible

	management.	with different large language models.
Agents	Agents is an open-source library for building autonomous language agents. It supports features like long-short term memory, tool usage, web navigation, multi-agent communication, human-agent interaction, and symbolic control.	- Memory Management: Handles both long-term and short-term memory. - Tool and API Integration: Enables agents to use external tools. - Web Navigation: Allows agents to browse and interact with web content. - Multi-Agent Communication: Facilitates interaction between agents. - Human-Agent Interaction: Supports collaborative work with humans. - Symbolic Reasoning: Incorporates logical reasoning capabilities.
BMTools	BMTools serves dual purposes: enhancing language models with external tool integration, and offering an open ecosystem where users can develop and exchange these capability-expanding tools.	- Tool Integration: Allows LLMs to use external tools for extended capabilities. - Community Platform: Encourages sharing and collaboration on tool development. - Functionality Enhancement: Expands the practical applications of LLMs.
crewAI	This AI agent framework prioritizes developer experience,	- Engineer-Focused: Tailored to the needs of

	offering engineers an accessible way to build sophisticated automations and agent-based practical applications	engineering tasks. - Simplified Development: Streamlines the creation of agents. - Automation: Facilitates automating repetitive or complex tasks.
Phidata	Phidata is a toolkit for building AI assistants using function calling. It enables developers to create AI assistants that perform tasks by invoking predefined functions.	- Function Calling Mechanisms: Simplifies task execution through functions. - AI Assistant Development: Focused on building helpful assistants. - API Integration: Connects with external APIs for extended functionality.

Table 6 main features of agent tools**17.9 References**

- [1] Z. Wang and et al. , "Data Management For Large Language Models: A Survey," arXiv:2312.01700v2, 2023.
- [2] H. ZHAO and et al., "Explainability for Large Language Models: A Survey," *ACM Trans. Intell. Syst. Technol.*, vol. 15, no. 2, p. 20, 2024.
- [3] J. Xu and et al, *The Future and FinTech: ABCDI and Beyond*, J. Xu, Ed., World Scientific Press, 2022.

- [4] A. McMahon, *Machine Learning Engineering with Python: Manage the lifecycle of machine learning models using MLOps with practical examples*, 2nd ed., Packt, 2023.
- [5] M. Steyvers, H. Tejada and et al., "What large language models know and what people think they know," *Nature Machine Intelligence*, vol. 7, p. 221–231, 2025.